

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №15

Тема: Швидке сортування QuickSort (Сортування Хоара)

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Опанувати принцип роботи алгоритму швидкого сортування (QuickSort), запропонованого Тоні Хоаром, розібрати схему розбиття Ломуто, навчитися аналізувати найкращий, середній і найгірший випадки складності залежно від вибору опорного елемента (півота), дослідити на практиці ефект рандомізованого вибору півота для уникнення найгіршого випадку.

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal).

Короткі теоретичні відомості

1. Принцип роботи (розділай і володарюй). QuickSort — рекурсивний алгоритм сортування, що працює за схемою "розділай і володарюй":

1. Обирається **опорний елемент (півот)** з масиву (чи його частини).
2. Виконується **розбиття (partition)**: масив переупорядковується так, щоб усі елементи, менші за півот, опинилися ліворуч від нього, а всі більші — праворуч; півот при цьому стає на своє остаточне місце в масиві.
3. Алгоритм **рекурсивно** застосовується до лівої та правої частин масиву (без півота).
4. Базовий випадок рекурсії — підмасив довжиною 0 або 1 (уже відсортований).

2. Схема розбиття Ломуто (Lomuto partition scheme). Обирає півотом останній елемент підмасиву. Заводиться індекс i межі "меншої за півот" області; для кожного елемента j від low до $high-1$, якщо $a[j] \leq pivot$, індекс i збільшується і елементи $a[i]$, $a[j]$ міняються місцями. Наприкінці півот ($a[high]$) міняється місцями з $a[i+1]$, що i є його остаточною позицією.

3. Аналіз складності:

- **Найкращий та середній випадки:** якщо півот щоразу ділить масив приблизно навпіл — глибина рекурсії $O(\log n)$, а на кожному рівні виконується $O(n)$ операцій розбиття, тому загальна складність **$O(n \log n)$** .
- **Найгірший випадок:** якщо півот щоразу виявляється мінімальним або максимальним елементом підмасиву (наприклад, при виборі півота як останнього елемента на **вже відсортованому масиві**), розбиття щоразу дає підмасиви розміром 0 і $n-1$ — глибина рекурсії стає $O(n)$, а загальна складність **деградує до $O(n^2)$** . Додатково це створює ризик переповнення стека викликів через лінійну глибину рекурсії.

4. Стратегії уникнення найгіршого випадку:

- **Рандомізований вибір пивота** — перед розбиттям випадковий елемент підмасиву міняється місцями з останнім, після чого застосовується звичайна схема Ломуто. Це робить імовірність постійного потрапляння в найгірший випадок експоненційно малою, незалежно від початкового порядку вхідних даних.
- **Медіана трьох (median-of-three)** — як пивот обирається медіана серед першого, середнього й останнього елементів підмасиву, що також суттєво знижує ймовірність незбалансованого розбиття на майже відсортованих даних.

5. Властивості QuickSort: середня складність $O(n \log n)$ з дуже малою прихованою константою (на практиці часто швидше за MergeSort і HeapSort), виконується **"на місці"** (in-place, потребує лише $O(\log n)$ додаткової пам'яті на стек рекурсії при збалансованому розбитті), **нестійкий** (relative order елементів з однаковими ключами не гарантується через далекі обміни при розбитті).

Хід виконання

Завдання 1. Ручна трасировка одного кроку розбиття (партиціонування)

Дано підмасив: [8, 3, 7, 1, 9, 2, 6] з пивотом — останнім елементом (6, схема Ломуто). Виконати вручну алгоритм розбиття, показавши рух індексів i та j .

Порядок виконання:

1. Зафіксуйте пивот = 6 (останній елемент), встановіть $i = \text{low} - 1 = -1$.
2. Для кожного j від 0 до 5 (усі елементи, крім пивота) порівняйте $a[j]$ з пивотом (6): якщо $a[j] \leq 6$, збільшіть i на 1 і поміняйте місцями $a[i]$ та $a[j]$.
3. Складіть таблицю зі стовпцями "j", " $a[j]$ ", " $a[j] \leq \text{pivot?}$ ", "новий i", "масив після дії" — заповніть по одному рядку для кожного j від 0 до 5.
4. Після завершення циклу поміняйте місцями $a[i+1]$ (перший елемент, що виявився більшим за пивот) і $a[\text{high}]$ (сам пивот) — це i є фінальна позиція пивота.
5. Запишіть отриманий масив після розбиття та позицію, на якій опинився пивот 6; переконайтесь, що всі елементи ліворуч від нього ≤ 6 , а всі праворуч > 6 .

Завдання 2. Повна трасировка QuickSort через дерево рекурсії

Дано масив: [8, 3, 7, 1, 9, 2, 6, 4]. Виконати вручну повне сортування алгоритмом QuickSort зі схемою Ломуто (пивот — завжди останній елемент поточного підмасиву), зобразивши дерево рекурсивних викликів.

Порядок виконання:

1. Виконайте розбиття всього масиву (аналогічно завданню 1), визначте позицію першого пивота і два підмасиви — лівий і правий.
2. Намалюйте вузол дерева рекурсії: "виклик QuickSort(весь масив)" → результат розбиття (позиція пивота, лівий підмасив, правий підмасив).
3. Рекурсивно повторіть крок розбиття окремо для лівого підмасиву і окремо для правого підмасиву, щоразу додаючи нові вузли до дерева рекурсії.

4. Продовжуйте, доки всі підмасиви не стануть довжиною 0 або 1 (листові вузли дерева).
5. Пройдіть деревом рекурсії "знизу вгору", зібравши остаточний відсортований масив, і переконайтесь, що він збігається з результатом сортування `sorted()`.
6. Порахуйте загальну глибину побудованого дерева рекурсії (максимальну кількість рівнів) і порівняйте з теоретичною оцінкою $\log_2 8 = 3$ для збалансованого випадку.

Завдання 3. Програмна реалізація QuickSort зі схемою Ломута

Реалізувати мовою Python рекурсивний алгоритм QuickSort зі схемою розбиття Ломута (півот — останній елемент), із підрахунком кількості порівнянь і максимальної глибини рекурсії.

Порядок виконання:

1. Напишіть допоміжну (внутрішню) функцію `_quicksort_lomuto(a, low, high, counters, depth)`, яка оновлює максимальну глибину рекурсії `counters["max_depth"]`.
2. Якщо `low < high`: виконайте розбиття (partition) за схемою Ломута, підраховуючи порівняння `a[j] <= pivot` у циклі.
3. Після розбиття отримайте позицію півота `p` і рекурсивно викличте функцію для `(low, p-1)` та `(p+1, high)` з `depth+1`.
4. Напишіть функцію-обгортку `quicksort_lomuto(arr)`, що ініціалізує лічильники й викликає внутрішню функцію.
5. Перевірте роботу алгоритму на тестовому масиві, порівнявши результат з `sorted()`.

Приклад реалізації мовою Python:

```
python
def quicksort_lomuto(arr):
    a = arr.copy()
    counters = {"comparisons": 0, "max_depth": 0}
    _quicksort_lomuto(a, 0, len(a)-1, counters, 0)
    return a, counters["comparisons"], counters["max_depth"]

def _quicksort_lomuto(a, low, high, counters, depth):
    counters["max_depth"] = max(counters["max_depth"], depth)
    if low < high:
        pivot = a[high]
        i = low - 1
        for j in range(low, high):
            counters["comparisons"] += 1
            if a[j] <= pivot:
                i += 1
                a[i], a[j] = a[j], a[i]
        a[i+1], a[high] = a[high], a[i+1]
        p = i + 1
        _quicksort_lomuto(a, low, p-1, counters, depth+1)
        _quicksort_lomuto(a, p+1, high, counters, depth+1)

test = [8, 5, 2, 9, 1, 7, 3, 6, 4]
result, c, depth = quicksort_lomuto(test)
print("Результат:", result, " порівнянь:", c, " макс. глибина рекурсії:", depth)
assert result == sorted(test)
```

Результат виконання програми:

Результат: [1, 2, 3, 4, 5, 6, 7, 8, 9] порівнянь: 19 макс.глибина рекурсії: 4

Завдання 4. Реалізація рандомізованого пивота та демонстрація найгіршого випадку

Реалізувати QuickSort з рандомізованим вибором пивота, порівняти кількість порівнянь і глибину рекурсії наївної (pivot — останній елемент) та рандомізованої версій на **вже відсортованому масиві** (найгірший випадок для наївної версії), побудувати графік.

Порядок виконання:

1. Напишіть функцію `quicksort_randomized(arr)`: перед розбиттям кожного підмасиву випадковим чином оберіть індекс `rnd` у діапазоні `[low, high]` за допомогою `random.randint(low, high)` і поміняйте місцями `a[rnd]` та `a[high]`; далі виконайте звичайну схему Ломуто.
2. Використовуйте вкладену функцію (замикання) для зручного накопичення лічильників порівнянь і глибини рекурсії без передачі їх явно через кожен виклик.
3. Для кількох розмірів $n \in \{500, 1000, 2000, 4000\}$ згенеруйте **вже відсортований** масив `list(range(n))` — це найгірший випадок для наївного QuickSort з пивотом-останнім-елементом.
4. Виконайте на кожному такому масиві обидві версії алгоритму (наївну й рандомізовану), зафіксувавши кількість порівнянь і максимальну глибину рекурсії для кожної.
5. Занесіть результати в таблицю "n — наївний(порівнянь) — наївний(глибина) — рандомізований(порівнянь) — рандомізований(глибина)".
6. Побудуйте графік залежності кількості порівнянь від n для обох версій на відсортованих даних.
7. Проаналізуйте отримані результати: як співвідноситься глибина рекурсії наївної версії з розміром масиву n (це є прямим індикатором переходу в режим $O(n^2)$), і наскільки рандомізація пивота вирішує цю проблему.

Приклад реалізації мовою Python:

```
python
import random

def quicksort_randomized(arr):
    a = arr.copy()
    counters = {"comparisons": 0, "max_depth": 0}
    def _qs(low, high, depth):
        counters["max_depth"] = max(counters["max_depth"], depth)
        if low < high:
            rnd = random.randint(low, high)
            a[rnd], a[high] = a[high], a[rnd]
            pivot = a[high]
            i = low - 1
            for j in range(low, high):
                counters["comparisons"] += 1
                if a[j] <= pivot:
                    i += 1
                    a[i], a[j] = a[j], a[i]
            a[i+1], a[high] = a[high], a[i+1]
            p = i + 1
            _qs(low, p-1, depth+1)
            _qs(p+1, high, depth+1)
    _qs(0, len(a)-1, 0)
    return a, counters["comparisons"], counters["max_depth"]

# Порівняння на ВІДСОРТОВАНОМУ масиві (найгірший випадок для наївного пивота)
for n in [500, 1000, 2000, 4000]:
    arr_sorted = list(range(n))
    r1, c1, d1 = quicksort_lomuto(arr_sorted.copy())
```

```

r2, c2, d2 = quicksort_randomized(arr_sorted.copy())
print(f"n={n:5d}: наївний: порівнянь={c1:>8}, глибина={d1:>5} | "
      f"рандомізований: порівнянь={c2:>8}, глибина={d2:>3}")

```

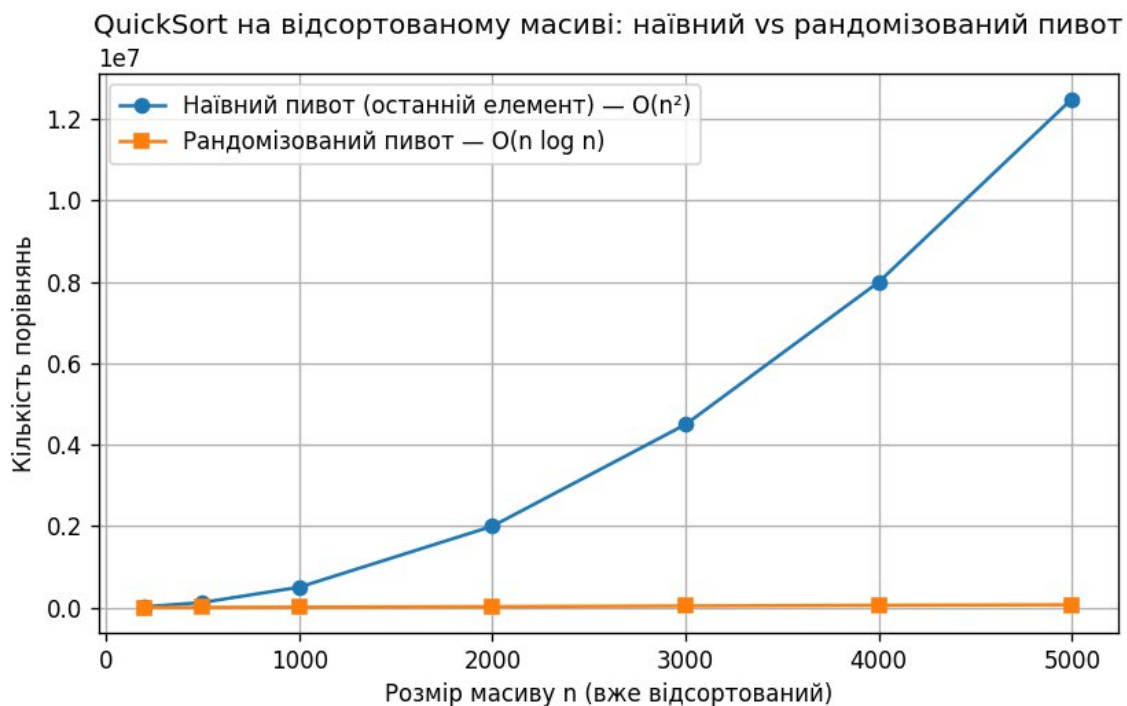
Результат виконання програми:

```

n= 500: наївний: порівнянь= 124750, глибина= 499 | рандомізований: порівнянь=  4716, глибина=  15
n= 1000: наївний: порівнянь= 499500, глибина= 999 | рандомізований: порівнянь= 10855, глибина=  20
n= 2000: наївний: порівнянь= 1999000, глибина= 1999 | рандомізований: порівнянь= 24892, глибина=  24
n= 4000: наївний: порівнянь= 7998000, глибина= 3999 | рандомізований: порівнянь= 55799, глибина=  26

```

Графік порівняння:



Висновок за результатами: на вже відсортованому масиві наївна реалізація з пивотом-останнім-елементом демонструє класичний найгірший випадок: глибина рекурсії **точно дорівнює $n-1$** (для $n=4000$ – глибина 3999!), що на практиці означало б переповнення стека викликів для великих масивів, а кількість порівнянь зростає квадратично (7998000 при $n=4000$, точно відповідає $n(n-1)/2$).

Рандомізований вибір пивота повністю усуває цю проблему: глибина рекурсії залишається логарифмічною ($\sim 24-26$ для $n=4000$, що узгоджується з $\log_2 4000 \approx 12$, помножене на невелику константу через ймовірнісний характер розбиття), а кількість порівнянь зростає за законом $n \log n$ – приблизно у 143 рази менше порівнянь, ніж у наївної версії при $n=4000$. Це підтверджує критичну важливість рандомізації (або медіани трьох) пивота на практиці: без неї QuickSort вразливий саме до тих вхідних даних (уже відсортованих або майже відсортованих), які найчастіше зустрічаються в реальних задачах.

Контрольні питання

1. Опишіть загальну схему алгоритму QuickSort ("розділяй і володарюй").
2. Поясни́ть принцип роботи схеми розбиття Ломуто.
3. За яких умов QuickSort досягає складності $O(n \log n)$, а за яких деградує до $O(n^2)$?
4. Чому пивот-останній-елемент на вже відсортованому масиві є найгіршим випадком для QuickSort?
5. Яку проблему вирішує рандомізований вибір пивота? Чи гарантує він повне уникнення найгіршого випадку?
6. Що таке стратегія "медіана трьох" і чим вона відрізняється від рандомізації?
7. Чи є QuickSort стійким алгоритмом сортування? Чи виконується він "на місці"?
8. Яка складність QuickSort за пам'яттю (глибина стека рекурсії) у середньому та найгіршому випадках?

Зміст звіту

1. Тема, мета роботи.
2. Розгорнуті ручні трасировки (завдання 1, 2), включно з деревом рекурсії.
3. Тексти програм та результати їх роботи (завдання 3, 4).
4. Графік порівняння наївного й рандомізованого пивота.
5. Висновки.